

Wie erkennt ein Computer handgeschriebene Ziffern?

Stefano Belloni

2026

Contents

1	Bilder, Computer und Bilderkennung	5
1.1	Was ist Bilderkennung?	5
1.1.1	Beispiele aus dem Alltag	5
1.2	Wie speichert ein Computer Bilder?	6
1.2.1	Experiment 1: Ein einfaches Schwarzweiß-Bild	6
1.2.2	Experiment 2: Graustufenbilder	7
1.2.3	Experiment 3: Farbbilder (RGB)	7
1.2.4	Größere Bilder	8
1.3	Der MNIST-Datensatz	8
1.3.1	Wie speichert der Computer ein MNIST-Bild?	8
2	Praktische Probleme	11
2.1	Das Problem der Handschrifterkennung	11
2.1.1	Warum ist das schwierig?	11
2.1.2	Naive Lösungsansätze	11
2.1.3	Das eigentliche Problem	13
2.1.4	Die Idee des Maschinellen Lernens	13
2.2	Ein erster Klassifikator: Durchschnittsbilder	14
2.2.1	Die durchschnittlichen Ziffern	15
2.2.2	Eine unbekannte Ziffer erkennen	15
2.3	Ein lernender Klassifikator: k-Nearest Neighbors (k-NN)	15
2.3.1	Die Grundidee	16
2.3.2	Wie funktioniert k-NN?	16
2.3.3	Abstände berechnen	17
2.3.4	Die Mehrheitsentscheidung	17
2.3.5	Warum funktioniert dieses Verfahren?	18
2.3.6	Vorteile und Nachteile	18
2.3.7	Vergleich mit dem Durchschnittsbild-Klassifikator	18
2.3.8	Warum verwenden wir dann neuronale Netze?	19
3	Künstliche Neuronen	21
3.1	Vom Vergleich zum Lernen: Das Perzeptron	21
3.1.1	Das Perzeptron	21
3.1.2	Das eigentliche Problem: Welche Gewichte?	22
3.1.3	Eine Verlustfunktion misst den Fehler	22
3.1.4	Die Suche nach dem Minimum	23
3.1.5	Der Gradient zeigt die Richtung	23
3.1.6	Vom Perzeptron zum neuronalen Netz	24
3.1.7	Warum brauchen neuronale Netze nichtlineare Funktionen?	24
3.1.8	Lineare Entscheidungsgrenzen und die Rolle der Aktivierungsfunktion	27
3.2	Deep Learning: Viele Schichten lernen komplexe Muster	29

3.2.1	Von einem Neuron zum neuronalen Netzwerk	30
3.2.2	Was bedeutet "Deep"?	30
3.2.3	Wie lernt ein tiefes Netzwerk Bilder zu verstehen?	31
3.2.4	Warum funktionieren tiefe Netzwerke so gut bei Bildern?	31
3.3	Convolutional Neural Networks (CNNs)	32
3.3.1	Die Grundidee einer Faltung	32
3.3.2	Feature Maps	33
3.3.3	Warum sind CNNs besser für Bilder geeignet?	33
3.3.4	Pooling: Wichtige Informationen behalten	34
3.3.5	Aufbau eines CNN für MNIST	35
3.3.6	Zusammenfassung	36
4	Experimente	37
4.1	Experiment 1: Einfluss der Aktivierungsfunktion	37
4.1.1	Fragestellung	37
4.1.2	Hintergrund	37
4.1.3	Experiment	38
4.1.4	Ergebnisse	38
4.1.5	Interpretation	38
4.2	Experiment 2: Wie tief muss ein Netzwerk sein?	38
4.2.1	Fragestellung	38
4.2.2	Hypothese	38
4.2.3	Experiment	39
4.2.4	Ergebnisse	39
4.2.5	Interpretation	39
4.3	Experiment 3: Einfluss der Lernrate	39
4.3.1	Fragestellung	39
4.3.2	Experiment	39
4.3.3	Interpretation	40
4.4	Experiment 4: Dropout und Überanpassung	40
4.4.1	Fragestellung	40
4.4.2	Idee von Dropout	40
4.4.3	Experiment	40
4.4.4	Interpretation	40
4.5	Experiment 5: DENSE Netzwerk gegen CNN	40
4.5.1	Fragestellung	40
4.5.2	Vergleich	41
4.5.3	Ergebnisse	41
4.5.4	Interpretation	41
4.6	Zusammenfassung der Experimente	41

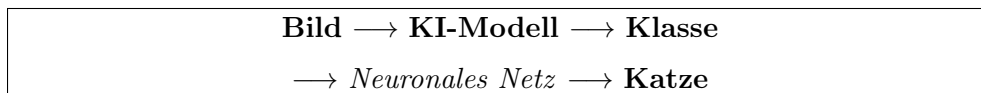
Chapter 1

Bilder, Computer und Bilderkennung

1.1 Was ist Bilderkennung?

Bilderkennung (auch *Image Recognition* oder *Computer Vision*) ist die Fähigkeit eines Computers, Bilder zu analysieren und zu verstehen. Ähnlich wie wir Menschen mit unseren Augen und unserem Gehirn erkennen Computer Muster, Formen, Farben und Objekte in Bildern.

Im Kern ist Bilderkennung häufig ein **Klassifikationsproblem**. Das bedeutet: Der Computer erhält ein Bild und soll entscheiden, zu welcher Kategorie es gehört.



Dabei kann es sich um einfache Klassen handeln, zum Beispiel:

- Katze oder Hund?
- Apfel oder Banane?
- Verkehrsschild oder Fußgänger?

Moderne KI-Systeme können jedoch nicht nur zwischen zwei Klassen unterscheiden, sondern aus Tausenden verschiedener Kategorien die wahrscheinlichste auswählen. Das Modell berechnet dabei für jede mögliche Klasse eine Wahrscheinlichkeit und wählt die Klasse mit der höchsten Wahrscheinlichkeit aus.

Damit ein Computer diese Aufgabe lösen kann, muss er zunächst aus vielen Beispielen lernen. Je mehr passende Trainingsdaten vorhanden sind, desto besser kann das Modell später unbekannte Bilder erkennen.

1.1.1 Beispiele aus dem Alltag

Bilderkennung begegnet uns heute in vielen Bereichen des täglichen Lebens:

- **Smartphone:** Das Telefon entsperrt sich durch Gesichtserkennung (z. B. Face ID).
- **Social Media:** Instagram oder Facebook können Personen auf Fotos automatisch erkennen und markieren.
- **Google Fotos:** Fotos werden automatisch nach Personen, Tieren oder Objekten sortiert und können später leicht gefunden werden.
- **Autonome Fahrzeuge:** Kameras erkennen Fahrbahnmarkierungen, andere Fahrzeuge, Fußgänger und Verkehrsschilder.

- **Medizin:** KI unterstützt Ärztinnen und Ärzte dabei, Krankheiten auf Röntgen- oder MRT-Bildern frühzeitig zu erkennen.
- **Einzelhandel:** Kameras zählen Kundinnen und Kunden oder analysieren Laufwege im Geschäft.

Alle diese Anwendungen basieren auf derselben Grundidee: Eine Künstliche Intelligenz lernt aus vielen Beispielen, typische Muster in Bildern zu erkennen und diese anschließend einer passenden Klasse zuzuordnen.

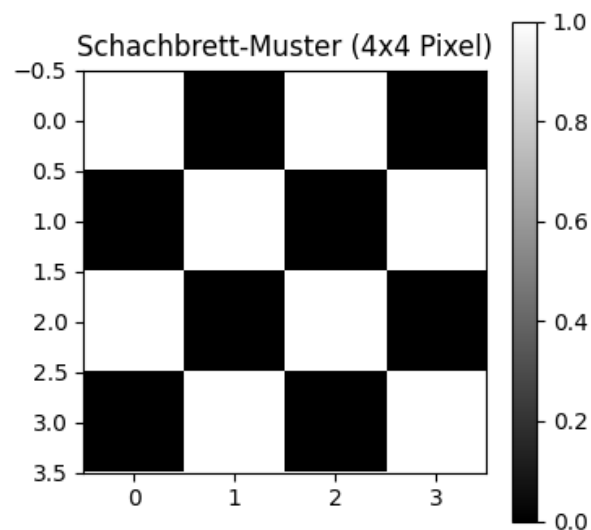
1.2 Wie speichert ein Computer Bilder?

Ein Bild auf dem Computer ist nichts anderes als eine **große Tabelle von Zahlen**, die man in der Mathematik als **Matrix** bezeichnet.

Ein Computer sieht also **keine Farben, keine Formen und keine Objekte**. Für ihn besteht ein Bild ausschließlich aus Zahlen.

1.2.1 Experiment 1: Ein einfaches Schwarzweiß-Bild

Stellen wir uns ein sehr kleines Bild mit einer Größe von nur **4 × 4 Pixeln** vor. Ein **Pixel** (englisch: *Picture Element*) ist der kleinste Baustein eines digitalen Bildes.



Darstellung des Bildes

So speichert der Computer das Bild

$$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

Dabei bedeutet:

- 1 = schwarzes Pixel
- 0 = weißes Pixel

Für den Computer ist dieses Bild also lediglich eine Matrix aus Nullen und Einsen.

1.2.2 Experiment 2: Graustufenbilder

Schwarzweißbilder mit verschiedenen Helligkeiten werden als **Graustufenbilder** gespeichert.

Jedes Pixel erhält einen Wert zwischen 0 und 255:

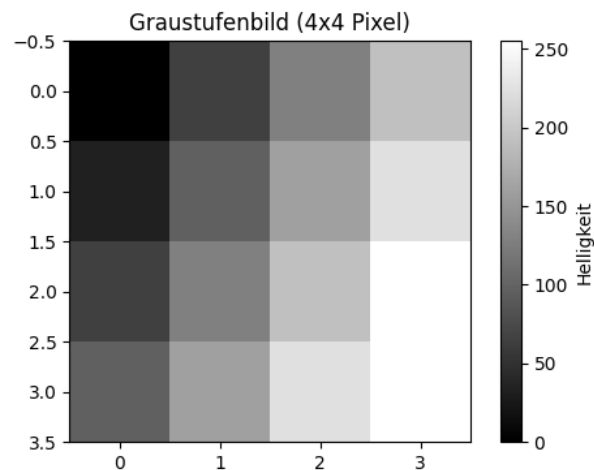
- 0 = schwarz
- 128 = mittleres Grau
- 255 = weiß

Ein mögliches 4×4 Bild könnte folgendermaßen aussehen:

$$\begin{bmatrix} 0 & 64 & 128 & 192 \\ 32 & 96 & 160 & 224 \\ 64 & 128 & 192 & 255 \\ 96 & 160 & 224 & 255 \end{bmatrix}$$

Je größer der Zahlenwert ist, desto heller erscheint das Pixel.
Mathematisch schreibt man:

$$\text{Helligkeit} \in [0, 255].$$



1.2.3 Experiment 3: Farbbilder (RGB)

Farbbilder verwenden das **RGB-Farbmodell**. RGB steht für:

- Rot (Red)
- Grün (Green)
- Blau (Blue)

Jedes Pixel besitzt deshalb nicht nur einen, sondern **drei Zahlenwerte**.

Mathematisch:

$$\text{Pixel}(i, j) = (R_{i,j}, G_{i,j}, B_{i,j}), \quad R, G, B \in [0, 255].$$

Einige Beispiele:

Rot	(255, 0, 0)
Grün	(0, 255, 0)
Blau	(0, 0, 255)
Weiß	(255, 255, 255)
Schwarz	(0, 0, 0)

Man kann sich ein Farbbild als drei übereinanderliegende Matrizen vorstellen: eine für Rot, eine für Grün und eine für Blau.

1.2.4 Größere Bilder

Ein echtes Foto besteht aus sehr viel mehr Pixeln als die kleinen Beispiele zuvor.

Format	Auflösung	Anzahl Pixel	Zahlen pro Graustufenbild
HD	1920×1080	ca. 2,1 Mio.	ca. 2,1 Mio.
4K	3840×2160	ca. 8,3 Mio.	ca. 8,3 Mio.
12 MP Smartphone	ca. 4000×3000	12 Mio.	12 Mio.

Bei einem Farbbild werden für jedes Pixel drei Zahlen gespeichert (Rot, Grün und Blau). Ein Smartphone-Foto mit 12 Millionen Pixeln besteht daher aus insgesamt etwa **36 Millionen Zahlen**.

Merke:

Ein Computer verarbeitet Bilder nicht als Fotos, sondern als große Matrizen aus Zahlen. Genau diese Zahlen werden später von Verfahren der Künstlichen Intelligenz analysiert.

1.3 Der MNIST-Datensatz

In diesem Kurs arbeiten wir mit dem bekannten **MNIST-Datensatz**. Er enthält handgeschriebene Ziffern und wird seit vielen Jahren verwendet, um Verfahren der Bilderkennung und des Maschinellen Lernens zu testen.

Jedes Bild zeigt genau eine handgeschriebene Ziffer zwischen 0 und 9. Da die Ziffern von vielen verschiedenen Personen geschrieben wurden, sehen sie alle etwas unterschiedlich aus. Genau diese Unterschiede machen die Aufgabe für eine Künstliche Intelligenz interessant.

Eigenschaft	Wert
Anzahl Bilder	70 000
Trainingsbilder	60 000
Testbilder	10 000
Bildgröße	28×28 Pixel
Pixel pro Bild	784
Bildtyp	Graustufen (0–255)
Klassen	Ziffern 0 bis 9

1.3.1 Wie speichert der Computer ein MNIST-Bild?

Ein MNIST-Bild besitzt eine Größe von 28×28 Pixeln. Intern wird das Bild als Matrix gespeichert:

$$\underbrace{28 \times 28}_{=784 \text{ Pixel}}$$

Jedes Pixel besitzt einen Grauwert zwischen 0 (schwarz) und 255 (weiß).

Für viele Berechnungen wird diese Matrix anschließend zu einem langen Vektor mit insgesamt 784 Zahlen umgeformt:

$$\begin{bmatrix} 0 & 0 & \dots & 255 \\ \vdots & \ddots & & \vdots \\ 30 & 120 & \dots & 0 \end{bmatrix} \longrightarrow (0, 0, 0, 30, 100, \dots, 255).$$

Der Computer erkennt dabei keine Ziffer, sondern verarbeitet lediglich diese Zahlen.



Chapter 2

Praktische Probleme

2.1 Das Problem der Handschrifterkennung

Stellen wir uns vor, wir ziehen zufällig eine handgeschriebene Ziffer aus einer Kiste und stellen die Frage:

„Welche Ziffer ist auf diesem Bild zu sehen?“

Für Menschen ist diese Aufgabe meist sehr einfach. Ein Computer muss dagegen lernen, wie er aus den Pixelwerten eines Bildes auf die richtige Ziffer schließen kann.

Die Aufgabe der Bilderkennung besteht also darin, jedem Bild genau eine der zehn Klassen

$$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

zuzuordnen.

2.1.1 Warum ist das schwierig?

Es gibt nicht die eine „richtige“ Schreibweise einer Ziffer. Jeder Mensch schreibt Zahlen etwas anders.

Unterschiede können zum Beispiel entstehen durch:

- unterschiedliche Handschriften,
- verschiedene Größen,
- leichte Drehungen oder Verschiebungen,
- unterschiedlich dicke Stifte,
- unterschiedlich stark geschriebene Linien.

Obwohl alle diese Bilder beispielsweise eine „7“ darstellen, unterscheiden sich ihre Pixelwerte teilweise deutlich. Für den Computer sehen sie daher zunächst wie völlig verschiedene Muster aus.

2.1.2 Naive Lösungsansätze

Bevor wir Künstliche Intelligenz verwenden, überlegen wir uns einige einfache Lösungsansätze.

Idee 1: Feste Regeln aufstellen

Eine andere Idee wäre, jede Ziffer durch feste Merkmale zu beschreiben.

Beispielsweise könnte man sagen:

„Eine 7 besitzt oben eine waagerechte Linie und darunter eine schräge Linie.“

Auch dieser Ansatz hat Probleme:

- Manche Menschen schreiben eine 7 ohne waagerechte Linie.
- Andere Ziffern besitzen ähnliche Linien.
- Begriffe wie „oben“, „schräg“ oder „lang“ sind schwer mathematisch zu definieren.

Schon wenige Ausnahmen führen dazu, dass solche Regeln unzuverlässig werden.

Idee 2: Vergleich durch zusammengefasste Merkmale

Eine andere Idee wäre, ein Bild nicht Pixel für Pixel zu vergleichen, sondern einige wichtige Eigenschaften des Bildes zusammenzufassen.

Zum Beispiel könnten wir für jede Ziffer bestimmte Werte berechnen:

- die durchschnittliche Helligkeit aller Pixel,
- die Anzahl der dunklen Pixel,
- die Position des dunkelsten Bereichs,
- die Breite und Höhe der geschriebenen Ziffer,
- die Anzahl von Linien oder Kanten.

Damit würde ein Bild nicht mehr durch 784 einzelne Pixelwerte beschrieben, sondern durch eine kleine Anzahl von Merkmalen.

Beispielsweise könnte eine Ziffer so dargestellt werden:

$$\text{Bild} \longrightarrow \begin{pmatrix} \text{durchschnittliche Helligkeit} \\ \text{Anzahl dunkler Pixel} \\ \text{Breite} \\ \text{Höhe} \end{pmatrix}$$

Der Merkmalsraum Wenn wir nur zwei Merkmale verwenden, können wir jede Ziffer als einen Punkt in einem zweidimensionalen Raum darstellen.

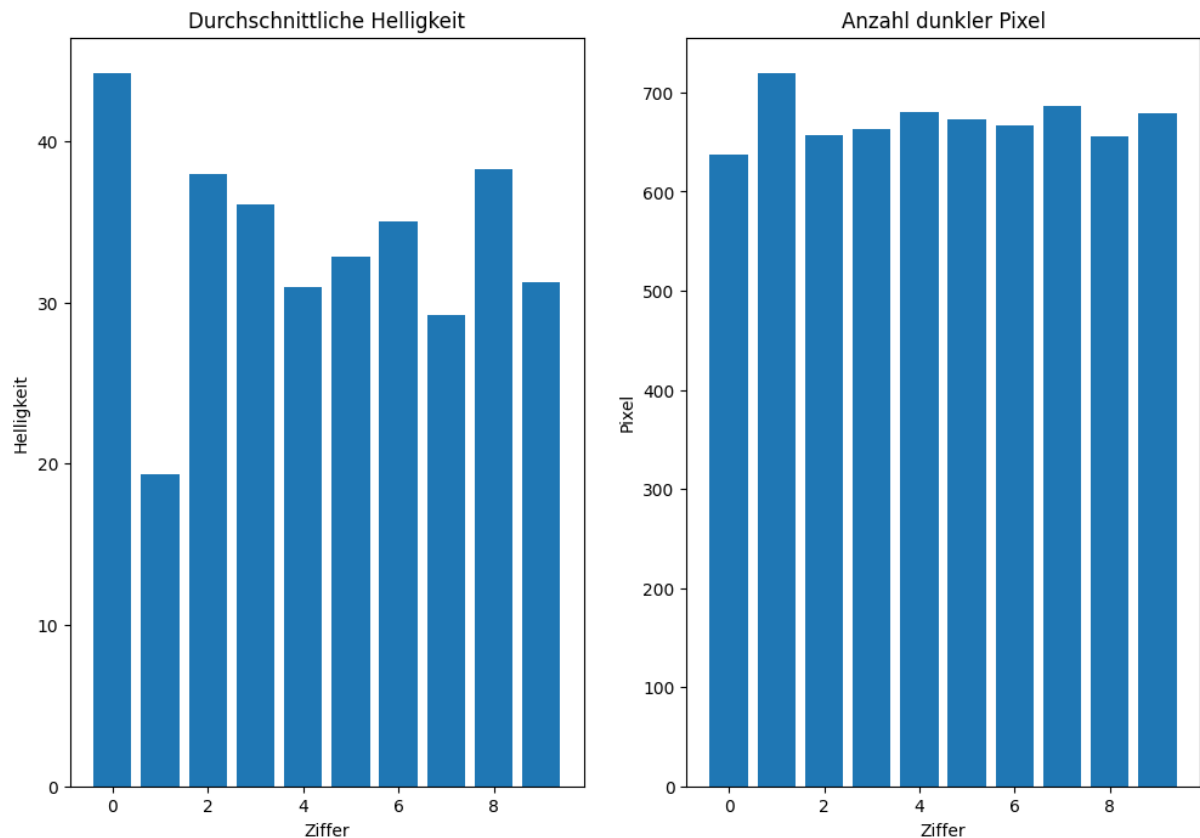
$$\text{Ziffer} \longrightarrow (\text{horizontaler Schwerpunkt}, \text{vertikaler Schwerpunkt})$$

Dadurch entsteht ein sogenannter **Merkmalsraum**.

Man erkennt jedoch auch die Grenzen dieser Methode: Einige Ziffern liegen sehr nah beieinander. Eine "3" und eine "8" können beispielsweise einen ähnlichen Schwerpunkt besitzen, obwohl ihre Formen völlig unterschiedlich sind.

Die Position eines Bildes im Merkmalsraum enthält also einige Informationen, aber nicht genug, um alle Ziffern zuverlässig zu erkennen.

Das Problem dabei ist jedoch: Diese Merkmale reichen nicht aus, um alle Ziffern zuverlässig zu unterscheiden.



Zwei verschiedene Ziffern können ähnliche Durchschnittswerte besitzen. Eine dick geschriebene "1" kann beispielsweise eine ähnliche Anzahl dunkler Pixel haben wie eine dünne "7". Auch die Position der Pixel allein beschreibt nicht die genaue Form einer Ziffer.

Außerdem ist es schwierig, die richtigen Merkmale von Hand auszuwählen:

- Welche Eigenschaften sind wirklich wichtig?
- Welche Merkmale unterscheiden eine "3" von einer "8"?
- Welche Merkmale funktionieren auch bei einer neuen Handschrift?

Die Form einer Ziffer besteht aus vielen kleinen Details und Zusammenhängen zwischen den Pixeln. Ein paar zusammengefasste Zahlenwerte können diese komplexen Muster nicht vollständig beschreiben.

2.1.3 Das eigentliche Problem

Die eigentliche Herausforderung lautet:

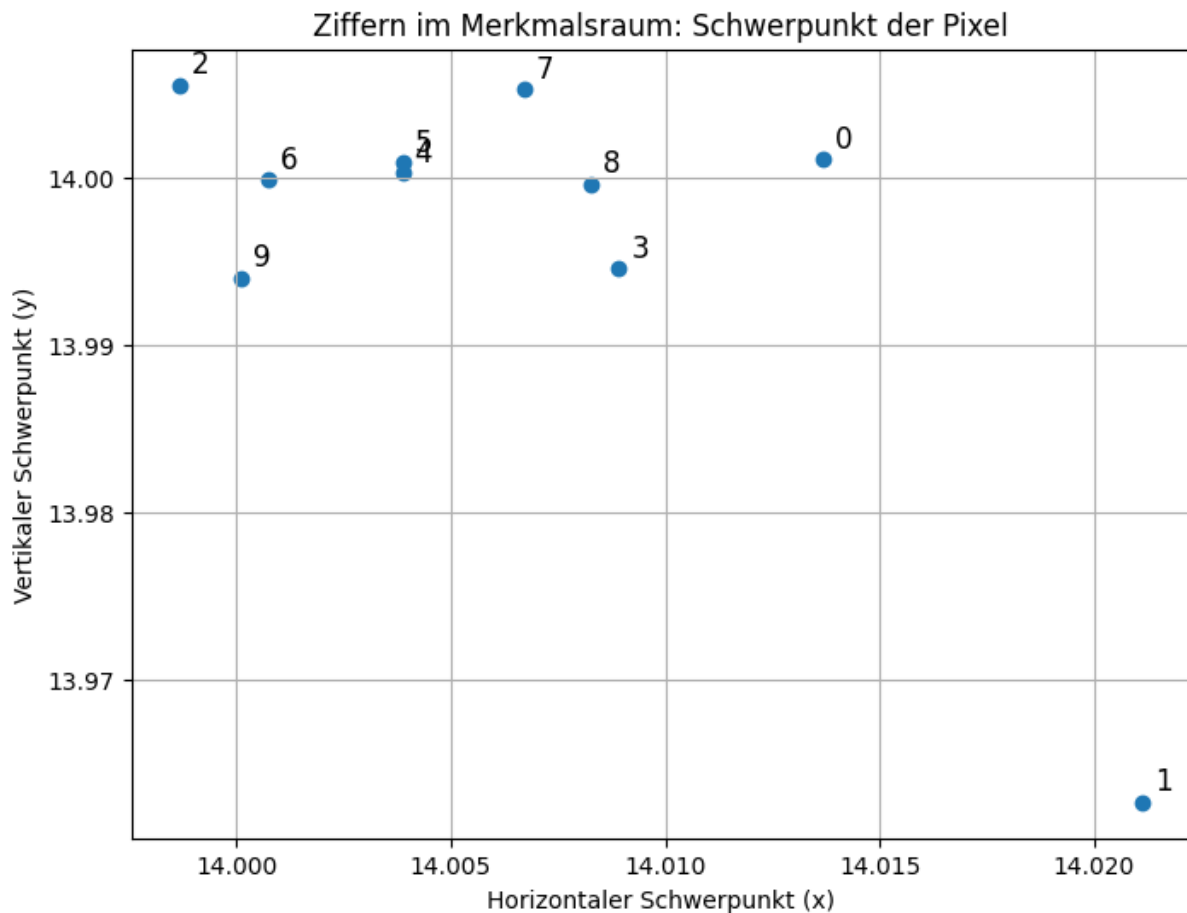
Wie findet man Regeln, die verschiedene Ziffern zuverlässig unterscheiden können?

Diese Regeln sind sehr kompliziert. Sie müssen gleichzeitig viele Eigenschaften eines Bildes berücksichtigen und trotzdem auch bei bisher unbekannten Handschriften funktionieren.

Menschen könnten solche Regeln kaum vollständig von Hand aufschreiben.

2.1.4 Die Idee des Maschinellen Lernens

Anstatt alle Regeln selbst zu formulieren, verfolgen wir einen anderen Ansatz:



Wir zeigen dem Computer viele bereits beschriftete Beispiele und lassen ihn die Regeln selbst lernen.

Der Computer erhält also tausende Bilder zusammen mit der richtigen Lösung:

Bild $\rightarrow$ richtige Ziffer

Aus diesen Beispielen erkennt das Modell selbst typische Muster und Zusammenhänge. Genau dieser Lernprozess bildet die Grundlage moderner Verfahren der Künstlichen Intelligenz und der Bilderkennung.

2.2 Ein erster Klassifikator: Durchschnittsbilder

Bevor wir Künstliche Intelligenz einsetzen, überlegen wir uns einen sehr einfachen Klassifikator. Die Idee lautet:

Für jede Ziffer berechnen wir das durchschnittliche Bild aller Trainingsbeispiele.

Da der MNIST-Datensatz viele tausend Beispiele jeder Ziffer enthält, kann man für jede Pixelposition den Mittelwert berechnen.

Für die Ziffer 3 erhält man beispielsweise

$$\text{Durchschnittsbild} = \frac{1}{N} \sum_{i=1}^N \text{Bild}_i.$$

Das Ergebnis ist kein scharfes Bild mehr, sondern eine Art *typische* oder *durchschnittliche* Ziffer.

2.2.1 Die durchschnittlichen Ziffern

Im Notebook berechnen wir den Mittelwert aller Bilder jeder Klasse.

Dabei entsteht für jede Ziffer genau ein Prototyp.

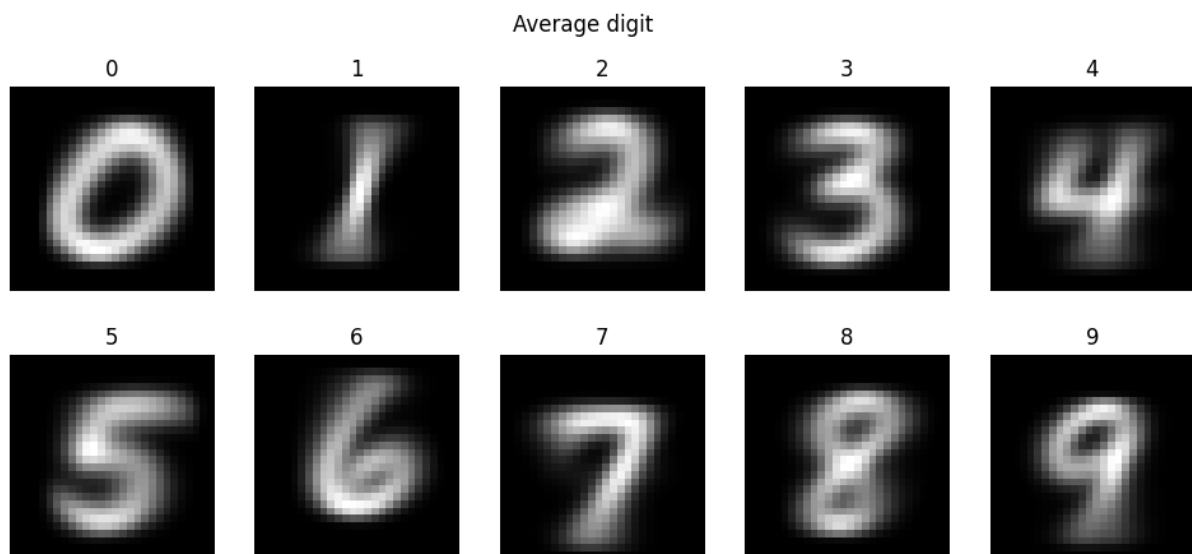


Figure 2.1: Die durchschnittlichen Ziffern des MNIST-Datensatzes.

Man erkennt, dass einige Ziffern sehr deutlich erscheinen, während andere etwas verschwommen wirken. Das liegt daran, dass verschiedene Menschen dieselbe Ziffer unterschiedlich schreiben.

2.2.2 Eine unbekannte Ziffer erkennen

Nun vergleichen wir ein neues Bild mit jedem Durchschnittsbild.

Dazu berechnen wir den Abstand zwischen den Pixelwerten.

Je kleiner der Abstand, desto ähnlicher sind sich die beiden Bilder.

Der Computer berechnet also

$$d_0, d_1, d_2, \dots, d_9$$

wobei d_i den Abstand zur durchschnittlichen Ziffer i beschreibt.

Die Vorhersage lautet dann

$$\text{Vorhersage} = \arg \min_i d_i$$

Das bedeutet:

Die Ziffer mit dem kleinsten Abstand wird als Ergebnis gewählt.

Dieses Verfahren ist erstaunlich einfach und funktioniert bereits überraschend gut. Trotzdem macht es noch viele Fehler, da nicht jede Handschrift dem Durchschnitt ähnelt.

2.3 Ein lernender Klassifikator: k-Nearest Neighbors (k-NN)

Unser erster Klassifikator hat für jede Ziffer genau ein Bild gespeichert: das Durchschnittsbild aller Trainingsdaten.

Dieses Verfahren ist sehr einfach und schnell. Allerdings geht dabei auch viel Information verloren. Jede Handschrift ist etwas anders und viele Besonderheiten verschwinden beim Bilden des Mittelwerts.

Wir stellen uns deshalb die Frage:

Warum speichern wir nicht einfach alle Trainingsbilder?

Genau diese Idee steckt hinter dem Verfahren **k-Nearest Neighbors (k-NN)**.

2.3.1 Die Grundidee

Anstatt nur zehn Durchschnittsbilder zu speichern, behalten wir alle Trainingsbilder.

Für den MNIST-Datensatz bedeutet das:

Durchschnittsbild-Klassifikator	k-NN
10 Bilder	60 000 Bilder

Der Computer besitzt dadurch viel mehr Beispiele und kann unbekannte Bilder besser mit bereits bekannten Handschriften vergleichen.

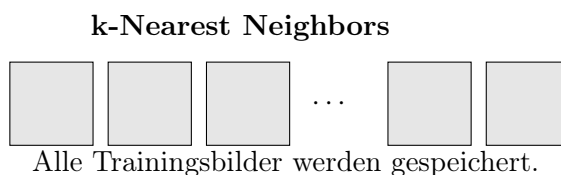
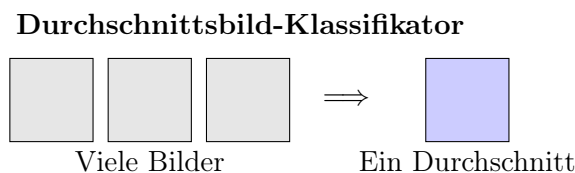


Figure 2.2: Der Durchschnittsbild-Klassifikator speichert nur einen Prototyp pro Klasse. Der k-NN-Klassifikator speichert dagegen alle Trainingsbilder.

2.3.2 Wie funktioniert k-NN?

Angenommen, wir möchten eine unbekannte handgeschriebene Ziffer klassifizieren.

Der Computer führt nun vier einfache Schritte aus.

1. Berechne den Abstand zu jedem Trainingsbild.
2. Sortiere alle Trainingsbilder nach ihrem Abstand.
3. Wähle die k Bilder mit dem kleinsten Abstand.
4. Die häufigste Ziffer gewinnt.

Der Name **k-Nearest Neighbors** bedeutet also:

- **Nearest** = die ähnlichsten Bilder

- **Neighbors** = die Nachbarn
- k = wie viele Nachbarn betrachtet werden

Ein typischer Wert ist beispielsweise

$$k = 5.$$

2.3.3 Abstände berechnen

Der wichtigste Schritt besteht darin, die Ähnlichkeit zweier Bilder zu berechnen.

Dazu betrachten wir jedes Pixel einzeln.

Sind zwei Bilder sehr ähnlich, dann unterscheiden sich ihre Pixelwerte nur wenig.

Mathematisch verwendet man häufig den sogenannten **euklidischen Abstand**

$$d(x, y) = \sqrt{\sum_{i=1}^{784} (x_i - y_i)^2}.$$

Dabei gilt:

- kleiner Abstand $\rightarrow$ Bilder sind sehr ähnlich
- großer Abstand $\rightarrow$ Bilder unterscheiden sich stark

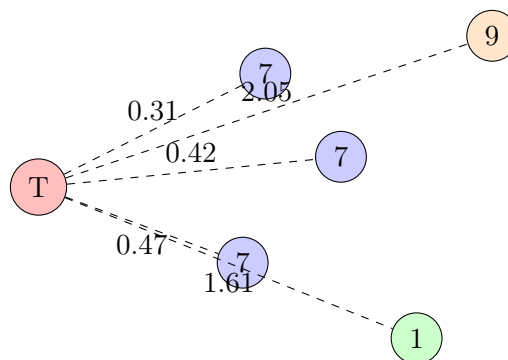


Figure 2.3: Der Abstand zwischen dem Testbild und allen Trainingsbildern wird berechnet. Kleine Abstände bedeuten hohe Ähnlichkeit.

Man erkennt sofort:

Die drei Bilder der Ziffer 7 besitzen die kleinsten Abstände.

Sie sehen dem Testbild also am ähnlichsten.

2.3.4 Die Mehrheitsentscheidung

Nachdem alle Abstände berechnet wurden, betrachten wir nur die k nächsten Nachbarn.

Nehmen wir an, wir erhalten folgende Klassen:

7, 7, 1, 7, 9

Nun zählen wir einfach, welche Ziffer am häufigsten vorkommt.

Klasse	Anzahl
7	3
1	1
9	1

Da die Ziffer 7 am häufigsten vorkommt, lautet die Vorhersage

7

Dieses Verfahren nennt man eine **Mehrheitsentscheidung** (*Majority Vote*).

2.3.5 Warum funktioniert dieses Verfahren?

Die Grundidee des k-NN-Verfahrens lautet:

Ähnliche Bilder gehören meistens zur gleichen Klasse.

Eine handgeschriebene 3 ähnelt normalerweise anderen handgeschriebenen 3en viel stärker als einer 8 oder einer 9.

Der Computer muss also keine komplizierten Regeln kennen.

Er nutzt lediglich die Erfahrung aus vielen bereits bekannten Beispielen.

Genau deshalb zählt k-NN bereits zu den Verfahren des **Maschinellen Lernens**.

2.3.6 Vorteile und Nachteile

Wie jedes Verfahren besitzt auch k-NN Vor- und Nachteile.

Vorteile	Nachteile
Sehr einfach zu verstehen	Alle Trainingsbilder müssen gespeichert werden.
Hohe Genauigkeit	Für jedes neue Bild müssen alle 60 000 Trainingsbilder verglichen werden.
Keine komplizierten Regeln notwendig	Die Vorhersage dauert dadurch relativ lange.

2.3.7 Vergleich mit dem Durchschnittsbild-Klassifikator

Der Durchschnittsbild-Klassifikator benötigt nur zehn Vergleiche.

Der k-NN-Klassifikator benötigt dagegen einen Vergleich mit jedem Trainingsbild.

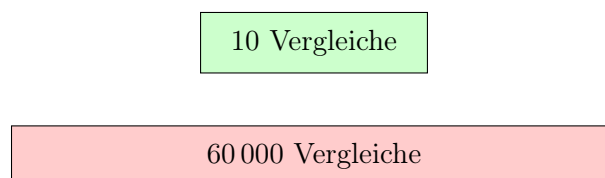


Figure 2.4: Vergleich des Rechenaufwands pro Vorhersage.

Der k-NN-Klassifikator ist also deutlich langsamer.

Dafür nutzt er alle vorhandenen Trainingsdaten und erreicht deshalb meist eine wesentlich höhere Genauigkeit.

2.3.8 Warum verwenden wir dann neuronale Netze?

Der Durchschnittsbild-Klassifikator ist sehr schnell, aber nicht besonders genau.

Der k-NN-Klassifikator ist deutlich genauer, benötigt jedoch für jede Vorhersage einen Vergleich mit allen Trainingsbildern.

Neuronale Netze kombinieren die Vorteile beider Verfahren.

Während des Trainings lernen sie aus allen Trainingsbildern eine mathematische Funktion.

Nach dem Training werden die ursprünglichen Bilder nicht mehr benötigt.

Dadurch können neuronale Netze neue Bilder sehr schnell klassifizieren und erreichen gleichzeitig eine sehr hohe Genauigkeit.

Chapter 3

Künstliche Neuronen

3.1 Vom Vergleich zum Lernen: Das Perzeptron

Bisher haben wir verschiedene Möglichkeiten kennengelernt, Bilder zu klassifizieren:

- Beim Durchschnittsbild-Klassifikator vergleichen wir ein Bild mit einem Prototyp.
- Beim k-Nearest-Neighbors-Verfahren vergleichen wir ein Bild mit vielen Beispielen.

Beide Methoden verwenden jedoch direkt die vorhandenen Bilder.

Die nächste Idee ist grundlegend anders:

Kann ein Computer selbst eine mathematische Regel lernen, die Bilder voneinander unterscheidet?

Genau diese Idee bildet die Grundlage neuronaler Netze.

3.1.1 Das Perzeptron

Das Perzeptron ist eines der einfachsten künstlichen Neuronen.

Es erhält mehrere Eingaben und berechnet daraus eine Ausgabe.

Für ein MNIST-Bild bestehen die Eingaben aus den Pixelwerten:

$$x_1, x_2, \dots, x_{784}$$

Jedes Pixel erhält ein eigenes Gewicht:

$$w_1, w_2, \dots, w_{784}$$

Die Gewichte bestimmen, wie wichtig ein bestimmtes Pixel für die Entscheidung ist.

Mathematisch berechnet das Perzeptron zunächst eine gewichtete Summe:

$$z = w_1x_1 + w_2x_2 + \dots + w_{784}x_{784} + b$$

Kurz geschrieben:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

Dabei bezeichnet:

- $\mathbf{x}$ die Pixelwerte des Bildes,
- $\mathbf{w}$ die gelernten Gewichte,

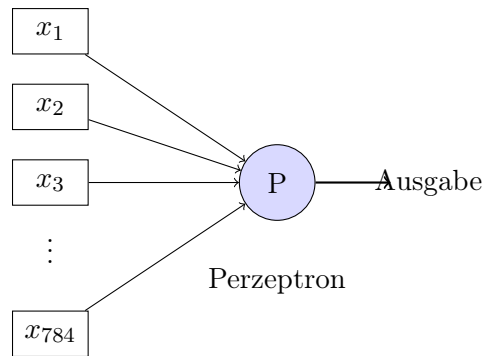


Figure 3.1: Ein Perzeptron kombiniert viele Eingaben zu einer Entscheidung.

- b den Bias (eine zusätzliche Verschiebung).

Danach entscheidet eine Aktivierungsfunktion über die Ausgabe.
Eine einfache Variante wäre:

$$y = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Damit kann ein Perzeptron beispielsweise entscheiden:

Ist dieses Bild eine 0 oder keine 0?

3.1.2 Das eigentliche Problem: Welche Gewichte?

Die zentrale Frage lautet:

Wie finden wir die richtigen Gewichte $w_1, w_2, \dots, w_{784}$?

Am Anfang kennen wir diese Werte nicht.

Wir beginnen deshalb mit zufälligen Gewichten:

$$w_i \approx \text{zufälliger Wert}$$

Das Modell macht Vorhersagen, vergleicht diese mit der richtigen Antwort und verbessert anschließend die Gewichte.

3.1.3 Eine Verlustfunktion misst den Fehler

Damit der Computer weiß, ob eine Änderung der Gewichte gut oder schlecht war, brauchen wir eine Fehlerfunktion.

Diese nennt man **Loss Function**.

Beispielsweise:

$$L(w) = (\text{Vorhersage} - \text{richtige Antwort})^2$$

Ein großer Fehler erzeugt einen großen Loss.

Das Ziel des Trainings ist:

Minimiere den Fehler

3.1.4 Die Suche nach dem Minimum

Stellen wir uns vor, der Fehler hängt nur von einem Gewicht w ab.

Dann entsteht eine Kurve:

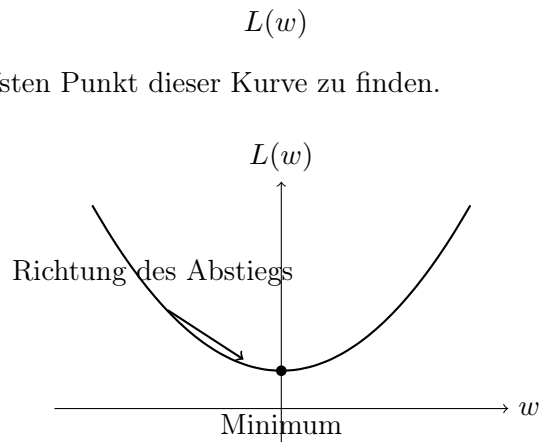


Figure 3.2: Die Verlustfunktion besitzt ein Minimum. Das Training sucht die Gewichte mit möglichst kleinem Fehler.

3.1.5 Der Gradient zeigt die Richtung

Die Steigung der Kurve sagt uns, in welche Richtung wir gehen müssen.

Diese Steigung nennt man **Gradient**.

Für ein einzelnes Gewicht:

$$\frac{\partial L}{\partial w}$$

bedeutet:

Wie verändert sich der Fehler, wenn wir das Gewicht ein kleines Stück verändern?

Ist die Steigung positiv:

$$\frac{\partial L}{\partial w} > 0$$

müssen wir das Gewicht kleiner machen.

Ist die Steigung negativ:

$$\frac{\partial L}{\partial w} < 0$$

müssen wir das Gewicht größer machen.

Der Lernschritt lautet:

$$w_{\text{neu}} = w_{\text{alt}} - \eta \frac{\partial L}{\partial w}$$

Der Parameter η heißt **Lernrate**.

Er bestimmt, wie groß die Schritte beim Lernen sind.

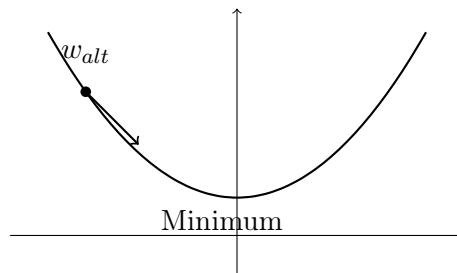


Figure 3.3: Der Gradient zeigt die Richtung, in der der Fehler kleiner wird.

3.1.6 Vom Perzeptron zum neuronalen Netz

Ein einzelnes Perzeptron kann nur sehr einfache Entscheidungen treffen.

Beispielsweise:

Ist dieses Bild eher eine 0 oder nicht?

Um komplexere Aufgaben zu lösen, verbinden wir viele Perzeptronen miteinander. Dadurch entstehen mehrere Schichten:

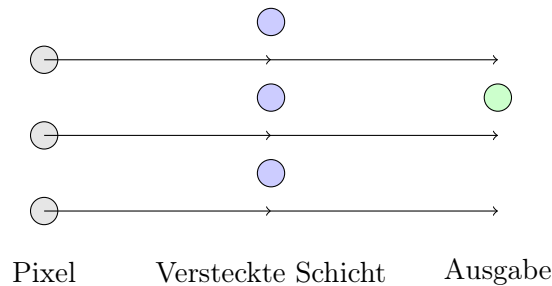


Figure 3.4: Ein neuronales Netz besteht aus vielen verbundenen Perzeptronen.

Ein modernes neuronales Netz lernt dadurch automatisch die wichtigen Merkmale eines Bildes:

Pixel $\rightarrow$ Linien $\rightarrow$ Formen $\rightarrow$ Ziffer

Genau diese Fähigkeit macht neuronale Netze so erfolgreich bei der Bilderkennung.

3.1.7 Warum brauchen neuronale Netze nichtlineare Funktionen?

Das Perzeptron besteht aus zwei Teilen:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

und einer Aktivierungsfunktion:

$$y = f(z)$$

Die Aktivierungsfunktion entscheidet, wie die Ausgabe des Neurons berechnet wird.

Das Problem einer rein linearen Aktivierung

Man könnte zunächst die Aktivierungsfunktion einfach weglassen:

$$y = z$$

Dann berechnet das Neuron nur eine lineare Funktion:

$$y = \mathbf{w} \cdot \mathbf{x} + b$$

Das scheint zunächst kein Problem zu sein.

Man könnte sogar mehrere Schichten miteinander verbinden:

Eingabe $\rightarrow$ Schicht 1 $\rightarrow$ Schicht 2 $\rightarrow$ Ausgabe

Aber mathematisch entsteht wieder nur eine einzige lineare Funktion.

Zum Beispiel:

$$y = W_2(W_1x + b_1) + b_2$$

kann umgeformt werden zu:

$$y = Wx + b$$

Die beiden Schichten bringen also keinen zusätzlichen Nutzen.

Egal wie viele lineare Schichten wir hinzufügen:

$$\text{linear} + \text{linear} + \text{linear} = \text{linear}$$

Das Netzwerk bleibt nur in der Lage, einfache gerade Grenzen zu lernen.

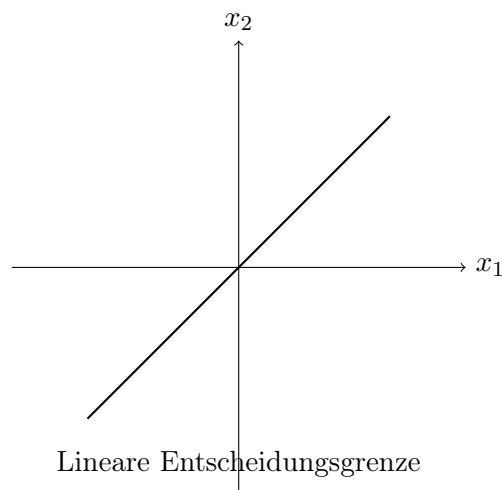


Figure 3.5: Ein Modell ohne Nichtlinearität kann nur gerade Trennlinien erzeugen.

Nichtlineare Aktivierungsfunktionen

Um komplexe Muster zu lernen, benötigen neuronale Netze nichtlineare Aktivierungsfunktionen.

Eine der ersten verwendeten Funktionen war die Sigmoid-Funktion:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sie verändert eine beliebige Zahl in einen Wert zwischen 0 und 1:

$$-\infty < x < \infty$$

wird zu

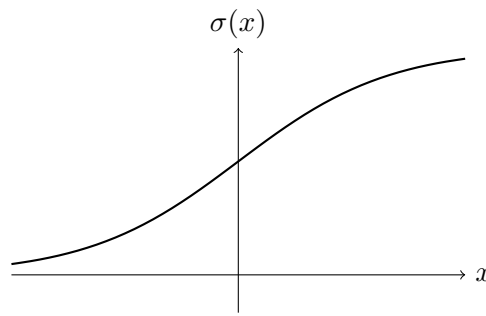


Figure 3.6: Die Sigmoid-Funktion erzeugt eine nichtlineare Transformation.

$$0 < \sigma(x) < 1$$

Heute wird häufig die ReLU-Funktion verwendet:

$$\text{ReLU}(x) = \max(0, x)$$

Sie ist sehr einfach:

$$\text{ReLU}(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

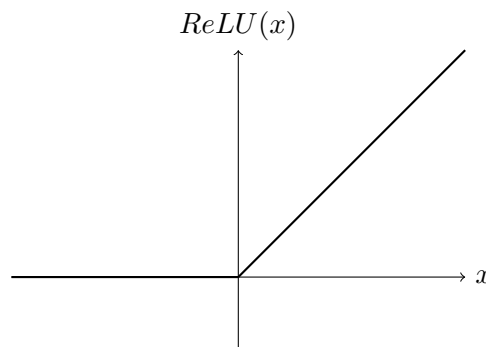


Figure 3.7: Die ReLU-Funktion ist für negative Werte null und für positive Werte linear.

Warum machen Nichtlinearitäten ein Netzwerk mächtiger?

Durch Aktivierungsfunktionen kann jede Schicht die Daten verändern.

Eine Schicht kann beispielsweise lernen:

Pixel $\rightarrow$ Kanten

Eine weitere Schicht:

Kanten $\rightarrow$ Formen

Eine weitere Schicht:

Formen $\rightarrow$ Ziffer

Ohne Nichtlinearitäten wären alle diese Schritte nur eine einzige große lineare Berechnung. Mit Nichtlinearitäten kann ein neuronales Netz dagegen komplizierte Strukturen lernen.

3.1.8 Lineare Entscheidungsgrenzen und die Rolle der Aktivierungsfunktion

Beim Perzeptron müssen wir zwischen zwei verschiedenen Konzepten unterscheiden:

1. die mathematische Berechnung innerhalb des Neurons,
2. die Form der Entscheidungsgrenze für die Klassifikation.

Das Perzeptron berechnet zunächst eine gewichtete Summe:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

oder kürzer:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

Diese Berechnung ist eine lineare Kombination der Eingaben.

Danach wird eine Aktivierungsfunktion angewendet:

$$y = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Diese Funktion nennt man **Schwellwertfunktion** oder **Sprungfunktion**.

Ist ein Perzeptron linear?

Die Sprungfunktion selbst ist mathematisch nicht linear.

Sie erzeugt eine harte Entscheidung:

$$\text{Eingabe} \rightarrow 0 \text{ oder } 1$$

Trotzdem wird das Perzeptron als **linearer Klassifikator** bezeichnet.

Der Grund ist die Form der Grenze zwischen den beiden Klassen.

Die Grenze entsteht dort, wo:

$$z = 0$$

also:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

Bei zwei Eingaben ergibt sich:

$$w_1x_1 + w_2x_2 + b = 0$$

Diese Gleichung beschreibt eine Gerade.

Für einfache Probleme reicht eine solche Grenze aus.

Beispielsweise:

Ist der Punkt links oder rechts von einer Geraden?

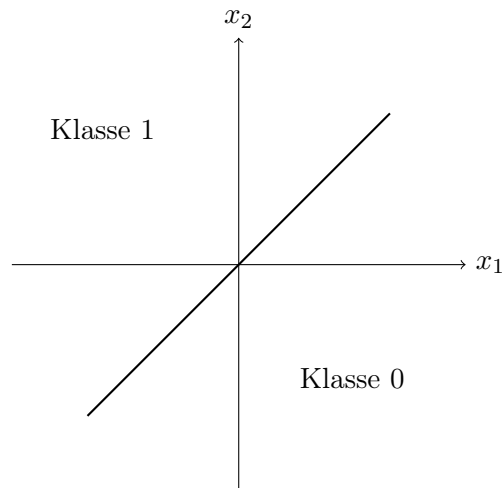


Figure 3.8: Ein Perzeptron kann Klassen durch eine lineare Grenze trennen.

Warum reicht eine lineare Grenze nicht für Bilder?

Bilder enthalten jedoch sehr komplexe Muster.

Bei Handschriften können Klassen beliebig geformte Bereiche im Merkmalsraum bilden. Eine einfache Gerade kann beispielsweise nicht das folgende Problem lösen:

XOR:

x_1	x_2	Klasse
0	0	0
0	1	1
1	0	1
1	1	0

Keine einzelne Gerade kann diese vier Punkte korrekt trennen.

Warum benötigen neuronale Netze Nichtlinearitäten?

Man könnte mehrere Perzeptronen hintereinander verbinden:

Schicht 1 $\rightarrow$ Schicht 2 $\rightarrow$ Schicht 3

Wenn jedoch jede Schicht nur lineare Berechnungen durchführt:

$$y = W_3(W_2(W_1x))$$

kann man die Matrizen zusammenfassen:

$$y = Wx$$

Das gesamte Netzwerk wäre wieder nur eine einzige lineare Funktion. Mehr Schichten würden also keinen zusätzlichen Nutzen bringen.

Linear + Linear + Linear = Linear

Damit neuronale Netze komplexe Muster lernen können, benötigen sie eine nichtlineare Aktivierungsfunktion.

Nichtlineare Aktivierungsfunktionen

Eine Aktivierungsfunktion verändert die Ausgabe eines Neurons:

$$a = f(z)$$

Beispiele sind:

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Die Ausgabe liegt zwischen 0 und 1:

$$0 < \sigma(z) < 1$$

ReLU Heute wird häufig die Funktion ReLU verwendet:

$$ReLU(z) = \max(0, z)$$

Sie lässt positive Werte unverändert und setzt negative Werte auf null.

Durch diese Nichtlinearität kann jede Schicht neue Eigenschaften lernen:

Pixel → Kanten
 → Formen
 → Ziffern

Die wichtige Erkenntnis lautet:

Ein einzelnes Perzeptron berechnet eine lineare Kombination von Eingaben. Die Nichtlinearität zwischen den Schichten ermöglicht es neuronalen Netzen, komplexe Muster zu lernen.

3.2 Deep Learning: Viele Schichten lernen komplexe Muster

Ein einzelnes Perzeptron kann nur sehr einfache Entscheidungen treffen.

Es kann beispielsweise lernen:

Ist ein Punkt links oder rechts von einer Linie?

Bei Bildern ist das Problem jedoch viel komplexer.

Eine handgeschriebene Ziffer besteht nicht aus einer einzigen Eigenschaft:

Pixel → Linien → Kurven → Formen → Ziffer

Die Idee von Deep Learning ist:

Statt dem Computer vorzugeben, welche Merkmale wichtig sind, lernt das Netzwerk diese Merkmale automatisch.

3.2.1 Von einem Neuron zum neuronalen Netzwerk

Ein künstliches Neuron berechnet:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

und anschließend:

$$a = f(z)$$

wobei f eine nichtlineare Aktivierungsfunktion ist.

Viele dieser Neuronen können miteinander verbunden werden.

Eine typische Struktur ist:

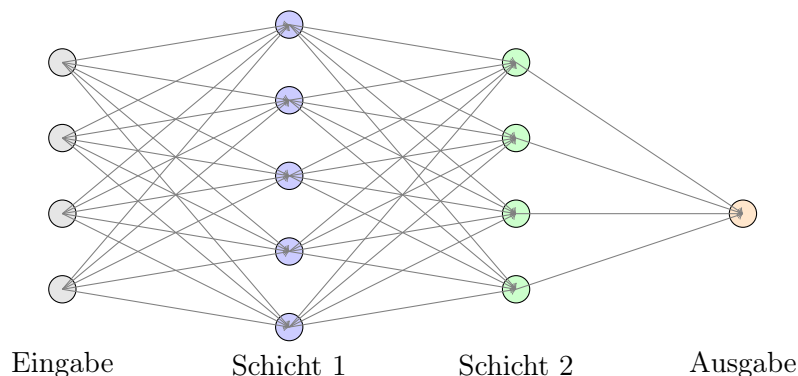


Figure 3.9: Ein künstliches neuronales Netzwerk besteht aus vielen verbundenen Neuronen.

Jede Verbindung besitzt ein eigenes Gewicht.

Ein Netzwerk mit mehreren Schichten besitzt deshalb sehr viele Parameter:

$$W_1, W_2, \dots, W_n$$

Beim Training werden diese Gewichte automatisch angepasst.

Der Lernprozess verwendet den Gradienten:

$$W_{\text{neu}} = W_{\text{alt}} - \eta \frac{\partial L}{\partial W}$$

Dieser Vorgang wird für alle Gewichte im Netzwerk wiederholt.

3.2.2 Was bedeutet "Deep"?

Das Wort **Deep** bedeutet nicht, dass das Netzwerk "intelligent" ist.

Es bedeutet lediglich:

$\text{Deep Learning} = \text{Lernen mit vielen Schichten}$

Ein Netzwerk mit:

$$\text{Eingabe} \rightarrow \text{eine Schicht} \rightarrow \text{Ausgabe}$$

ist ein einfaches neuronales Netz.

Ein Netzwerk mit vielen Zwischenschichten:

$$\text{Eingabe} \rightarrow \underbrace{\text{Schicht} \rightarrow \text{Schicht} \rightarrow \dots \rightarrow \text{Schicht}}_{\text{viele Ebenen}} \rightarrow \text{Ausgabe}$$

wird Deep Neural Network genannt.

3.2.3 Wie lernt ein tiefes Netzwerk Bilder zu verstehen?

Ein wichtiger Unterschied zu klassischen Verfahren ist:

Der Computer wählt die Merkmale selbst.

Bei einem traditionellen Ansatz müsste ein Mensch Merkmale definieren:

$$\text{Bild} \rightarrow \begin{cases} \text{Helligkeit} \\ \text{Kanten} \\ \text{Schwerpunkte} \\ \text{Formen} \end{cases} \rightarrow \text{Klassifikation}$$

Bei Deep Learning:

$$\text{Bild} \rightarrow \text{Neuronales Netzwerk} \rightarrow \text{Klassifikation}$$

Die ersten Schichten lernen häufig einfache Eigenschaften:

$$\text{Pixel} \rightarrow \text{Kanten}$$

Mittlere Schichten lernen komplexere Muster:

$$\text{Kanten} \rightarrow \text{Kurven und Formen}$$

Tiefe Schichten lernen abstrakte Konzepte:

$$\text{Formen} \rightarrow \text{Ziffer}$$

$$\text{Pixel} \longrightarrow \text{Kanten} \longrightarrow \text{Formen} \longrightarrow \text{Ziffer}$$

Figure 3.10: Tiefe Netzwerke lernen hierarchische Merkmale.

3.2.4 Warum funktionieren tiefe Netzwerke so gut bei Bildern?

Bilder besitzen eine natürliche Struktur:

- benachbarte Pixel hängen zusammen,
- Linien bilden Formen,
- Formen bilden Objekte.

Ein tiefes Netzwerk kann diese Hierarchie automatisch lernen.

Bei der Handschrifterkennung:

$$28 \times 28 = 784$$

Pixel werden als Eingabe verwendet.

Das Netzwerk lernt selbst:

$$784 \text{ Pixel} \rightarrow \text{Merkmale} \rightarrow 10 \text{ Klassen}$$

Die Ausgabe besteht aus zehn Wahrscheinlichkeiten:

$$P(0), P(1), \dots, P(9)$$

Beispiel:

Ziffer	Wahrscheinlichkeit
0	0.01
1	0.02
2	0.03
3	0.85
4	0.01
...	

Das Modell entscheidet:

höchste Wahrscheinlichkeit = Vorhersage

3.3 Convolutional Neural Networks (CNNs)

Ein normales neuronales Netzwerk (Dense Network) behandelt ein Bild als eine lange Liste von Zahlen:

$$28 \times 28 = 784$$

Pixel werden einfach zu einem Vektor umgeformt:

$$(784) \rightarrow (128) \rightarrow (64) \rightarrow (10)$$

Das Netzwerk weiß dabei nicht, dass bestimmte Pixel nebeneinander liegen.

Für ein Bild ist diese Information aber sehr wichtig:

- benachbarte Pixel bilden Linien,
- Linien bilden Formen,
- Formen bilden Objekte.

Ein CNN (Convolutional Neural Network) wurde speziell entwickelt, um diese räumliche Struktur von Bildern auszunutzen.

3.3.1 Die Grundidee einer Faltung

Eine Faltung (Convolution) untersucht kleine Ausschnitte eines Bildes.

Statt das gesamte Bild auf einmal zu betrachten, bewegt sich ein kleines Fenster über das Bild.

Dieses Fenster nennt man:

Filter oder Kernel

Beispiel:

$$3 \times 3$$

Filter:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

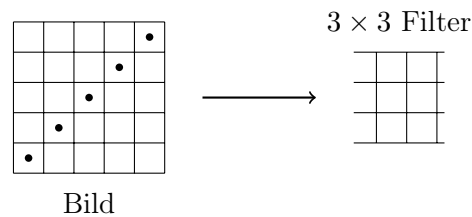


Figure 3.11: Ein Filter untersucht kleine Bereiche eines Bildes.

Dieser Filter könnte beispielsweise vertikale Kanten erkennen.
Für jeden Bildausschnitt wird berechnet:

$$\text{Ausgabe} = \sum_{i,j} \text{Pixel}_{i,j} \cdot \text{Filter}_{i,j}$$

Der Filter besitzt eigene Gewichte.

Diese Gewichte werden genauso wie bei einem normalen neuronalen Netzwerk durch Training gelernt.

3.3.2 Feature Maps

Wenn ein Filter über ein Bild bewegt wird, entsteht eine neue Darstellung:

$$\text{Bild} \rightarrow \text{Feature Map}$$

Eine Feature Map zeigt, wo ein bestimmtes Merkmal gefunden wurde.
Beispiele:

- ein Filter erkennt horizontale Linien,
- ein anderer Filter erkennt Kreise,
- ein anderer Filter erkennt Ecken.

Ein CNN verwendet viele Filter gleichzeitig:

$$\text{Bild} \rightarrow \begin{bmatrix} \text{Kanten} \\ \text{Kurven} \\ \text{Strukturen} \end{bmatrix}$$

3.3.3 Warum sind CNNs besser für Bilder geeignet?

Problem bei Dense Netzwerken

Ein Dense Netzwerk verbindet jedes Eingabepixel mit jedem Neuron:

$$784 \times 128 = 100352$$

Gewichte werden bereits für die erste Schicht benötigt.

Außerdem verliert das Netzwerk die Information über die Position:

$$\begin{array}{ccc} A & B & C \\ D & E & F \\ G & H & I \end{array}$$

und

$$\begin{array}{ccc} I & H & G \\ F & E & D \\ C & B & A \end{array}$$

enthalten dieselben Zahlen, aber eine völlig andere Struktur.

Vorteile von CNNs

CNNs verwenden lokale Verbindungen:

$$3 \times 3$$

statt:

$$28 \times 28$$

Dadurch entstehen mehrere Vorteile:

1. Weniger Parameter Ein Filter mit Größe:

$$3 \times 3$$

benötigt nur:

$$9$$

Gewichte.

Diese Gewichte werden über das gesamte Bild wiederverwendet.

2. Erkennen von Positionen Ein Filter für eine Kante funktioniert egal, ob die Kante links oder rechts im Bild erscheint.

Das nennt man:

Translation Invariance

3. Hierarchisches Lernen Mehrere CNN-Schichten lernen immer komplexere Eigenschaften:

Schicht 1 : Kanten

Schicht 2 : Kurven und einfache Formen

Schicht 3 : Ziffern und Objekte

3.3.4 Pooling: Wichtige Informationen behalten

Nach einer Faltung wird häufig eine Pooling-Schicht verwendet.

Beispiel:

$$2 \times 2$$

Max-Pooling:

$$\begin{bmatrix} 1 & 5 \\ 2 & 3 \end{bmatrix} \rightarrow 5$$

Die Idee:

- kleine Details werden entfernt,
- wichtige Merkmale bleiben erhalten,
- das Bild wird kleiner.

Dadurch wird das Netzwerk robuster gegenüber kleinen Verschiebungen.

3.3.5 Aufbau eines CNN für MNIST

Ein typisches CNN für die Handschrifterkennung:

$$28 \times 28 \times 1$$

↓

$$\text{Conv}(32)$$

↓

$$\text{MaxPooling}$$

↓

$$\text{Conv}(64)$$

↓

$$\text{Dense}(128)$$

↓

10 Klassen

Die letzte Schicht verwendet Softmax:

$$P(0), P(1), \dots, P(9)$$

Beispiel:

Ziffer	Wahrscheinlichkeit
0	0.01
1	0.02
2	0.03
3	0.91
4	0.01

Das Modell entscheidet:

Klasse mit größter Wahrscheinlichkeit

3.3.6 Zusammenfassung

Ein Dense Netzwerk lernt:

Pixel $\rightarrow$ Klasse

Ein CNN lernt:

Pixel $\rightarrow$ Kanten $\rightarrow$ Formen $\rightarrow$ Objekte $\rightarrow$ Klasse

Die wichtigste Idee:

CNNs sind neuronale Netzwerke, die die Struktur von Bildern ausnutzen und automatisch passende Merkmale lernen.

Chapter 4

Experimente

In diesem Kapitel untersuchen wir verschiedene Eigenschaften neuronaler Netze praktisch.

Wir verändern jeweils nur einen Teil des Modells und beobachten, wie sich das Verhalten verändert.

Dabei betrachten wir nicht nur die Genauigkeit, sondern auch:

- Trainingszeit,
- Anzahl der Parameter,
- Lernverlauf,
- Fehler bei der Klassifikation.

Das Ziel ist nicht nur ein möglichst gutes Modell zu bauen, sondern zu verstehen, warum bestimmte Architekturen besser funktionieren.

4.1 Experiment 1: Einfluss der Aktivierungsfunktion

4.1.1 Fragestellung

Welche Aktivierungsfunktion eignet sich am besten für ein neuronales Netz?

Wir vergleichen:

- ReLU,
- Sigmoid,
- Tanh.

4.1.2 Hintergrund

Ein Neuron berechnet zunächst:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

und anschließend:

$$a = f(z)$$

Die Aktivierungsfunktion bringt die notwendige Nichtlinearität in das Netzwerk. Ohne Aktivierungsfunktion wäre:

$$Linear + Linear + Linear = Linear$$

Das Netzwerk könnte dann keine komplexen Muster lernen.

4.1.3 Experiment

Wir verwenden dieselbe Architektur:

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$$

Nur die Aktivierungsfunktion wird verändert.

Figure 4.1: Vergleich verschiedener Aktivierungsfunktionen.

4.1.4 Ergebnisse

Die Modelle werden anhand folgender Werte verglichen:

- Testgenauigkeit,
- Trainingszeit,
- Lernkurve.

Figure 4.2: Vergleich der Genauigkeit verschiedener Aktivierungsfunktionen.

4.1.5 Interpretation

Typischerweise funktioniert ReLU besser als Sigmoid.

Der Grund:

- ReLU erzeugt stärkere Gradienten,
- Training mit vielen Schichten ist stabiler,
- Berechnung ist einfacher.

4.2 Experiment 2: Wie tief muss ein Netzwerk sein?

4.2.1 Fragestellung

Verbessert eine größere Anzahl von Schichten automatisch das Ergebnis?

4.2.2 Hypothese

Mehr Schichten können komplexere Muster lernen.

Allerdings können sehr große Netzwerke auch Probleme verursachen:

- längere Trainingszeit,
- mehr Parameter,
- Überanpassung.

4.2.3 Experiment

Wir vergleichen verschiedene Architekturen:

Modell A: $784 \rightarrow 32 \rightarrow 10$

Modell B: $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$

Modell C: $784 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10$

Figure 4.3: Verschiedene Netzwerkgrößen.

4.2.4 Ergebnisse

Figure 4.4: Einfluss der Netzwerkgröße auf die Genauigkeit.

4.2.5 Interpretation

Ein größeres Netzwerk besitzt mehr lernbare Parameter:

mehr Parameter $\rightarrow$ mehr Modellkapazität

Dies bedeutet aber nicht automatisch bessere Ergebnisse.

Das Netzwerk muss ausreichend Daten sehen und richtig trainiert werden.

4.3 Experiment 3: Einfluss der Lernrate

4.3.1 Fragestellung

Wie groß sollten die Schritte beim Lernen sein?

Beim Gradient Descent werden die Gewichte angepasst:

$$W_{neu} = W_{alt} - \eta \frac{\partial L}{\partial W}$$

Der Parameter η ist die Lernrate.

4.3.2 Experiment

Wir vergleichen:

$$\eta = 0.0001$$

$$\eta = 0.001$$

$$\eta = 0.01$$

Figure 4.5: Training mit verschiedenen Lernraten.

4.3.3 Interpretation

Eine zu kleine Lernrate:

→ Training sehr langsam

Eine zu große Lernrate:

→ Minimum wird übersprungen

Eine gute Lernrate ermöglicht schnelles und stabiles Lernen.

4.4 Experiment 4: Dropout und Überanpassung

4.4.1 Fragestellung

Kann ein Netzwerk zu viel auswendig lernen?

Ein Modell soll nicht nur die Trainingsdaten erkennen:

Training $\neq$ Realität

Es soll auf neue Bilder generalisieren.

4.4.2 Idee von Dropout

Während des Trainings werden zufällig einzelne Neuronen deaktiviert.

Figure 4.6: Dropout entfernt zufällig Verbindungen während des Trainings.

Dadurch lernt das Netzwerk robustere Eigenschaften.

4.4.3 Experiment

Vergleich:

$Dropout = 0$

$Dropout = 0.2$

$Dropout = 0.5$

Figure 4.7: Einfluss von Dropout auf die Generalisierung.

4.4.4 Interpretation

Dropout kann die Trainingsgenauigkeit reduzieren, aber die Testgenauigkeit verbessern.

4.5 Experiment 5: Dense Netzwerk gegen CNN

4.5.1 Fragestellung

Warum verwenden moderne Bilderkennungssysteme CNNs?

4.5.2 Vergleich

Dense Netzwerk:

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$$

CNN:

$$28 \times 28 \rightarrow \textit{Conv} \rightarrow \textit{Pooling} \rightarrow \textit{Dense} \rightarrow 10$$

Figure 4.8: Dense Netzwerk und CNN im Vergleich.

4.5.3 Ergebnisse

Wir vergleichen:

- Genauigkeit,
- Anzahl Parameter,
- Fehlerbilder.

Figure 4.9: Fehlervergleich zwischen Dense Netzwerk und CNN.

4.5.4 Interpretation

CNNs nutzen die Struktur von Bildern:

$$\textit{Pixel} \rightarrow \textit{Kanten} \rightarrow \textit{Formen} \rightarrow \textit{Objekte}$$

Dadurch können sie Bildinformationen effizienter verarbeiten.

4.6 Zusammenfassung der Experimente

Die Experimente zeigen:

- Aktivierungsfunktionen ermöglichen Nichtlinearität.
- Mehr Schichten erhöhen die Modellkapazität.
- Die Lernrate bestimmt die Geschwindigkeit des Trainings.
- Dropout verbessert die Generalisierung.
- CNNs sind für Bilder besser geeignet als normale Dense Netzwerke.

Die wichtigste Erkenntnis:

Gute künstliche Intelligenz entsteht nicht nur durch ein großes Modell, sondern durch die passende Kombination aus Daten, Architektur und Training.